



19. Mai 2026

Übungen zur Vorlesung Service-Basierte Komponenten-Architekturen SS 2026

Übungsblatt Nr. 4

(Abgabe bis: Dienstag, den 26. Mai 2026, 12:00 Uhr)

Aufgabe 1 (Case Study für Microservices, FA. WirSchiffenDas)

Das Unternehmen WirSchiffenDas GmbH & Co KGaA hat über viele Jahre hinweg eine *monolithische* Software-Plattform für den Vertrieb ihrer Produkte (Komponenten für Hochsee-Yachten) entwickelt. Man erkennt, dass Teile der resultierenden Software-Architektur *in die Jahre* gekommen sind und strebt nun Microservice-Architektur an. Dazu wurde nun durch eine erste Version entwickelt, die Sie *analysieren* sollten. Eine Baustein-Sicht dieser ersten Version einer Microservice-basierten Architektur nebst diversen Erläuterungen und Meinungen des Entwicklungsteams zu der Software-Architektur hat man ihnen bereitgestellt, Sie finden diese auf LEA.

Der zuständige Software-Architekt des Entwicklungsteams hat jedoch Zweifel an der Eignung dieser Architektur. Er vertraut ihnen die Aufgabe an, diese Architektur neu zu entwerfen. Man bittet Sie, auf Grundlage Ihrer Kenntnisse zu Microservices eine neue Software-Architektur zu modellieren.

Darüber hinaus ist man auch über weitere organisatorische Aspekte interessiert, die sich aus dieser Verbesserung der Software-Architektur ergeben. Dazu hat man ein paar Statements aus einem Meeting der Bereichsleitung des Bereichs „Technik“ zusammengetragen, wo vor kurzem der Ansatz Microservices kurz vorgestellt und *leidenschaftlich* diskutiert wurde.

Statement Nr. 1:

„Wir haben eine strenge Komponenten- und Schichtenbildung, entsprechend haben wir auch Komponenten- und Schichtenteams. Das kann nur Vorteile haben, das wird übernommen!“ Nach kurzer Diskussion beruht man auch für Microservices auf diese Einteilung und verspricht sich dadurch einen klaren Vorteil. Was denken sie dazu? Was würde sie dem Unternehmen eher empfehlen?

Statement Nr. 2:

„Das wird schon! Die Geschäftsführung des Unternehmens muss doch für so eine geringfügige Architekturänderung, also der Einführung einer Microservice-Architektur,

nicht informiert werden. Auch über die Integration von modernen KI-Support sieht unbedenklich“ Was denken sie über dieses Statement?

Statement Nr. 3:

„Wir sind cloud-ready und somit auch horizontal skalierbar!“

Da man die Anwendungsschicht auf *einer* Virtuellen Maschine (VM) eingesetzt hat und diese Instanz insgesamt als *einen* Microservice ansieht, resultiert man, dass dieser Microservice auch auf einen Cloud-Anbieter (z.B. Amazon WS) ausgelagert werden kann. Die Anwendung gilt somit als *skalierbar*! Es besteht eh keine Notwendigkeit der Skalierung einzelner Komponenten, so die Bereichsleitung. Wie stehen sie zu dieser Einschätzung?

Statement Nr. 4:

„Ach immer dieses API-Team...“. Das „API-Team“ (vgl. Baustein-Sicht) hat sich wohl in letzter Zeit oft über eine zunehmende Komplexität der Komponente „GP-API“ aus dem Presentation Logic Layer beschwert, gibt aber Begründungen nicht explizit an (-- man gibt zu, vielleicht auch nicht richtig hingehört zu haben...). Was für Gründe könnten das sein aus ihrer Sicht? Wie könnte man diese Schicht ggf. im Rahmen der Microservice-Architektur optimieren?

Statement Nr. 5:

„Die Analyse-Komponente ist DAS Problem überhaupt!“ Leider sorgt die Analyse-Komponente innerhalb der Komponente „Manufacturing Products“ häufig für Probleme während der Konstruktion von Produkten (siehe Erläuterungen in Dokument). Was sind hier konkret die Probleme? Wie sähe eine mögliche Lösungsstrategie aus?

Statement Nr. 6:

„Unsere langfristige Strategie: ein globales und universales Datenmodell!“ Da man auch immer noch diverse Bestandsysteme verwendet (SAP ERP, Oracle CRM), sieht man die Etablierung eines gemeinsamen universalen Datenmodells als unabdingbar. Was denken sie? Wie würden sie hier mit den heterogenen Datenmodellen umgehen? Wird diese Meinung von allen Entwicklerteams mitgetragen?

Statement Nr. 7:

„Man ist insgesamt sehr stolz auf moderne CI/CD-Lösung.“ Wie stehen Sie zu dieser Einschätzung?

Statement Nr. 8:

„Der Workflow „Order-Fulfillment“ läuft bestens und ist fachlich und software-technisch auf solidem Fundament gebaut“. Wie stehen Sie zu dieser Einschätzung?

Ihre Aufgaben:

a.)

Gehen sie auf die o.g. Statements kritisch ein und beantworten sie die aufgeführten Fragen in 2-3 Sätzen. Die resultierenden Erkenntnisse können sie natürlich *später* gerne in die Architektur einfließen lassen. Beachten sie dabei auch die Erläuterungen aus der Architekturbeschreibung.

Zur Diskussion der Vor- und Nachteile von Komponenten-, Schicht- und Funktionsteams empfehle ich den Artikel von Unterauer (2017), siehe SEKA-Literatur-Repository.

Zur Diskussion des Statements Nr. 4 empfehle ich folgenden Artikel von Sam Newmann:

<http://samnewman.io/patterns/architectural/bff/>
(Last Access: 19.05.2026)

b.)

Modellieren sie eine *erste Version* einer neuen Software-Architektur auf Basis des Architekturstils Microservices. Modellieren Sie eine Kontext-Sicht (inklusive einer Einbindung der Akteure) sowie eine Baustein-Sicht (vgl. Kapitel 3, Abschnitt 2). Falls schon anwendbar, können sie auch eine erste Verteilungssicht modellieren. Sie können sich bei der Baustein-Sicht gerne an die allgemeinen Erläuterungen zu Microservices aus Kapitel 2 (Abschnitt Microservices) sowie an die in Kapitel 3 (Abschnitt 4) orientieren.

c.) *nicht vergessen bzw. nicht übersehen ;-)*

Modellieren Sie ein fachliches Domänen-Modell gemäß des Domain-Driven-Design (Strategisches Design). Modellieren sie dazu eine *erste* Context Map (vgl. Kapitel 2 (Abschnitt Microservices)), um die Beziehungen zwischen den identifizierten fachlichen Kontexten („Bounded Context“) zu beschreiben.

d.)

Bei der Modellierung der Sichten sollten Sie sich zudem an die *abgeleiteten* Lösungen für *alle* Architecture Smells (Abschnitt IV.A) sowie für *alle* Anti Patterns (Abschnitt IV.B) aus dem Katalog von (Schirgi and Brenner, 2021) orientieren! Geben Sie zunächst für jedes Anti Pattern bzw. für jeden Architecture Smells eine *erste* Einschätzung ab, inwiefern diese in der bestehenden Architektur vorhanden sind. Beurteilen Sie dann auch, wie und ob die Lösungen für ein Anti Pattern bzw. für einen Architecture Smell für Ihre Lösung relevant sind. Berücksichtigen Sie auch die Smells und Anti Pattern, wenn diese nicht im Kontext von Microservices auftauchen (z.B. eher auf Komponenten-Ebene).

e.)

Entwickeln Sie eine erste kurze KI-Vision. Wo könnten KI-basierte Verfahren wie z.B. LLM / MCP das System sinnvoll unterstützen?